

Moodle Theme Developer Guide

Mustache · SCSS · Child Themes · PHP Renderers

Project	Ranees EdTech
Platform	Moodle 4.5 (Self-hosted, Docker)
Base Theme	Boost
Version	1.0

Table of Contents

- 1. Introduction to Moodle Theming**
 - What is a Moodle theme
 - Theme inheritance (parent/child)
 - The Boost theme as base
 - How Moodle compiles SCSS and renders templates
- 2. Child Theme File Structure**
 - Every file explained with purpose
- 3. Creating Your First Child Theme**
 - Step-by-step from zero to active theme
- 4. SCSS Customization**
 - pre.scss, post.scss, Bootstrap variables
- 5. Mustache Templates**
 - Syntax, overrides, key templates, data context
- 6. PHP Renderer Overrides**
 - core_renderer.php extension
- 7. JavaScript — AMD Modules**
 - RequireJS, creating modules, third-party libs
- 8. Development Workflow (Docker)**
 - Cache purging, designer mode, volume mounting
- 9. Common Mistakes**
- 10. Quick Reference**
 - Minimal file templates, URLs, CLI commands

1. Introduction to Moodle Theming

1.1 What is a Moodle Theme?

A Moodle theme is a plugin that controls the visual presentation of your Moodle site. It defines the HTML structure (via Mustache templates), styling (via SCSS/CSS), and JavaScript behaviour of every page students and teachers see.

Themes live in the **/theme/** directory of your Moodle installation. Each theme is a self-contained folder with a strict naming convention: lowercase letters and underscores only (e.g. **theme_ranees**).

1.2 Theme Inheritance — Parent / Child

Moodle themes support inheritance. A child theme declares one or more parent themes in its **config.php**. When Moodle looks for a template, SCSS, or renderer, it searches the child first, then walks up the parent chain.

This means your child theme only needs to contain the files you actually want to override — everything else is inherited automatically from the parent.

Layer	Who provides it	Override in child?
Bootstrap 4 SCSS	theme/boost	Yes — via pre.scss variables
Moodle core SCSS	theme/boost	Yes — via post.scss
HTML templates	theme/boost	Yes — copy & edit in templates/
PHP renderers	lib/outputrenderers.php	Yes — extend core_renderer
JavaScript	theme/boost/amd	Yes — add your own modules

1.3 The Boost Theme

Boost is Moodle's modern default theme, built on Bootstrap 4. All new custom themes should extend Boost. It provides:

- A responsive two-column layout with collapsible drawers
- Bootstrap 4 grid, components, and utilities
- Full SCSS source (not just compiled CSS)
- Mustache templates for every page region
- A clean AMD JavaScript module system

■ *Never edit files inside theme/boost/ directly. Your changes will be lost on every Moodle upgrade. Always create a child theme.*

1.4 How Moodle Compiles SCSS and Renders Templates

SCSS Compilation: Moodle compiles SCSS server-side using PHP (the ScssPhp library). The compilation order is:

- 1. Your pre.scss (variable overrides)
- 2. Bootstrap 4 source
- 3. Moodle core SCSS
- 4. Your post.scss (custom additions)

The compiled CSS is cached. You must purge caches after any SCSS change.

Template Rendering: When a page loads, Moodle's PHP layout controller (e.g. layout/drawers.php) builds a data context array and passes it to a Mustache template. The template is rendered server-side and sent to the browser as plain HTML.

2. Child Theme File Structure

Here is the complete file structure of a child theme named **ranees**. Files marked **REQUIRED** must exist for the theme to load.

```

theme/ranees/
├── config.php           ← REQUIRED: parents, layouts, SCSS config
├── version.php         ← REQUIRED: version number for upgrades
├── lib.php             ← REQUIRED: pre/post SCSS hook functions
├── settings.php        ← Optional: admin settings (logo, colour pickers)
├──
├── scss/
│   ├── pre.scss        ← Bootstrap variable overrides (loaded first)
│   └── post.scss       ← Custom CSS added after compilation
│   └──
├── templates/         ← Mustache HTML overrides (copy from boost)
│   ├── navbar.mustache
│   ├── login.mustache
│   ├── footer.mustache
│   └── drawers.mustache
│   └──
├── layout/            ← PHP layout controllers
│   └── drawers.php
│   └──
├── classes/
│   ├── output/
│   └── core_renderer.php ← PHP renderer overrides
│   └──
├── pix/
│   └── favicon.ico     ← Your site favicon
│   └──
├── lang/
│   ├── en/
│   └── theme_ranees.php ← Theme name string (REQUIRED)
│   └──
├── amd/
│   ├── src/           ← JS source (ES modules)
│   │   └── mycustom.js
│   └── build/         ← Compiled JS (auto-generated by Grunt)
│       └── mycustom.min.js

```

File	Purpose
config.php	Registers the theme: parent, layouts, SCSS entry points, regions
version.php	Theme version number — Moodle uses this to detect upgrades
lib.php	PHP functions that inject pre.scss and post.scss into the compiler
settings.php	Adds admin settings (logo upload, colour picker, font selector)
scss/pre.scss	Loaded before Bootstrap — override \$primary, \$font-family-base, etc.

scss/post.scss	Loaded after everything — add any custom CSS rules here
templates/*.mustache	HTML template overrides — only include templates you change
layout/*.php	Controls which template + context is used for each page type
classes/output/core_renderer.php	Override how Moodle generates specific HTML fragments
pix/	Images: favicon.ico, logo, screenshots
lang/en/theme_*.php	Language strings — minimum: \$string['pluginname'] = 'Your Theme'
amd/src/	JavaScript AMD modules — your custom JS goes here

3. Creating Your First Child Theme (Step by Step)

Step 1 — Create the Theme Directory

Inside your Moodle installation, create the theme folder. In your Docker setup, the Moodle source is at `/var/www/html/`.

```
docker exec -it moodle_app bash
mkdir -p /var/www/html/theme/ranees/scss
mkdir -p /var/www/html/theme/ranees/lang/en
mkdir -p /var/www/html/theme/ranees/pix
```

Step 2 — config.php

This is the most important file. It tells Moodle everything about your theme.

```
<?php
// theme/ranees/config.php

defined('MOODLE_INTERNAL') || die();

$THEME->name          = 'ranees';
$THEME->parents        = ['boost'];           // inherit from Boost
$THEME->sheets         = [];                  // no plain CSS – we use SCSS
$THEME->usefallback    = true;

// SCSS entry point functions from lib.php
$THEME->scss = function($theme) {
    return theme_ranees_get_main_scss_content($theme);
};

// Layouts – reuse Boost's layout files
$THEME->layouts = [
    'base'          => ['file' => 'drawers.php', 'regions' => ['side-pre']],
    'standard'      => ['file' => 'drawers.php', 'regions' => ['side-pre']],
    'course'        => ['file' => 'drawers.php', 'regions' => ['side-pre'], 'defaultregion' => 'side-pre'],
    'coursecategory' => ['file' => 'drawers.php', 'regions' => ['side-pre']],
    'incourse'      => ['file' => 'drawers.php', 'regions' => ['side-pre']],
    'frontpage'     => ['file' => 'drawers.php', 'regions' => ['side-pre'], 'defaultregion' => 'side-pre'],
    'admin'         => ['file' => 'drawers.php', 'regions' => ['side-pre']],
    'mydashboard'   => ['file' => 'drawers.php', 'regions' => ['side-pre'], 'defaultregion' => 'side-pre'],
    'login'         => ['file' => 'login.php', 'regions' => [], 'options' => ['nofooter' => true, 'nona'],
    'popup'         => ['file' => 'columns1.php', 'regions' => []],
    'embedded'      => ['file' => 'embedded.php', 'regions' => []],
    'maintenance'   => ['file' => 'maintenance.php', 'regions' => []],
    'print'         => ['file' => 'columns1.php', 'regions' => []],
    'redirect'      => ['file' => 'embedded.php', 'regions' => []],
    'report'        => ['file' => 'drawers.php', 'regions' => ['side-pre']],
    'secure'        => ['file' => 'secure.php', 'regions' => []],
];
```

```
$THEME->enable_dock = false;
$THEME->extrascsscallback = 'theme_ranees_get_extra_scss';
$THEME->prescsscallback = 'theme_ranees_get_pre_scss';
$THEME->scss_compiler_class = 'theme_boost\autoprefixer';
$THEME->rendererfactory = 'theme_overridden_renderer_factory';
$THEME->requiredblocks = '';
$THEME->addblockposition = BLOCK_ADDBLOCK_POSITION_FLATNAV;
```

Step 3 — version.php

```
<?php
// theme/ranees/version.php

defined('MOODLE_INTERNAL') || die();

$plugin->version = 2024042800; // YYYYMMDDXX format
$plugin->requires = 2023042400; // Minimum Moodle version (4.2)
$plugin->component = 'theme_ranees';
```

Step 4 — lib.php

These three functions are called by Moodle's SCSS compiler. They inject your SCSS into the compilation pipeline.

```
<?php
// theme/ranees/lib.php

defined('MOODLE_INTERNAL') || die();

/**
 * Returns the main SCSS content – loads Boost's preset as the base.
 */
function theme_ranees_get_main_scss_content($theme) {
    global $CFG;
    $scss = '';
    // Load Boost's main preset as base
    $filename = !empty($theme->settings->preset) ? $theme->settings->preset : null;
    $fs = get_file_storage();
    if ($filename && ($file = $fs->get_file(context_system::instance()->id,
        'theme_ranees', 'preset', 0, '/', $filename))) {
        $scss .= $file->get_content();
    } else {
        // Default to Boost's preset
        $scss .= file_get_contents($CFG->dirroot . '/theme/boost/scss/preset/default.scss');
    }
    return $scss;
}

/**
 * Returns SCSS to inject BEFORE Bootstrap compilation.
 * Use this to override Bootstrap variables.
 */
function theme_ranees_get_pre_scss($theme) {
```



```

        global $CFG;
        $scss = '';
        $scss .= file_get_contents($CFG->dirroot . '/theme/ranees/scss/pre.scss');
        return $scss;
    }

    /**
     * Returns SCSS to inject AFTER compilation.
     * Use this to add custom rules on top.
     */
    function theme_ranees_get_extra_scss($theme) {
        global $CFG;
        $scss = '';
        $scss .= file_get_contents($CFG->dirroot . '/theme/ranees/scss/post.scss');
        return $scss;
    }

```

Step 5 — Language File (Required)

```

<?php
// theme/ranees/lang/en/theme_ranees.php

$string['pluginname'] = 'Ranees EdTech';
$string['choosereadme'] = 'A custom theme for Ranees EdTech, built on Boost.';

```

Step 6 — Activate the Theme

Go to **Site administration** → **Appearance** → **Themes** → **Theme selector** and choose Ranees EdTech.

Or use the CLI:

```

docker exec moodle_app php /var/www/html/admin/cli/cfg.php \
    --name=theme --set=ranees

```

Step 7 — Purge Caches

```

docker exec moodle_app php /var/www/html/admin/cli/purge_caches.php

```

4.1 pre.scss — Override Bootstrap Variables

This is where you define your brand colours, fonts, and spacing.

[illegible]

4.2 post.scss — Custom CSS

post.scss is loaded after all Bootstrap and Moodle SCSS. Add your own rules here. You have full access to Bootstrap 4 variables and mixins.

4.3 Common Bootstrap 4 Variables Reference

Variable	Default	What it controls
\$primary	#007bff	Primary colour — buttons, links, focus rings

\$secondary	#6c757d	Secondary colour
\$font-family-base	system fonts	Body font family
\$font-size-base	1rem	Base font size (16px)
\$line-height-base	1.5	Body line height
\$border-radius	.25rem	Default border radius
\$grid-gutter-width	30px	Column gutter width
\$navbar-height	—	Navbar height
\$headings-font-weight	500	h1–h6 font weight
\$link-color	\$primary	Hyperlink colour
\$body-bg	#fff	Page background colour
\$body-color	#212529	Default text colour

5. Mustache Templates

5.1 Mustache Syntax

Moodle uses the Mustache templating language. Templates are plain HTML files with special tags for dynamic content.

Syntax	Meaning
{{variable}}	Output the value of a variable (HTML-escaped)
{{{variable}}}	Output raw HTML (not escaped — use carefully)
{{#block}} ... {{/block}}	Render if block is truthy, or loop if block is an array
{{^block}} ... {{/block}}	Render if block is falsy (inverse/else block)
{{>partial}}	Include another template (partial)
{{! comment }}	Comment — not rendered in output
{{#str}} key, component {{/str}}	Moodle string helper — outputs language string
{{#pix}} icon, component {{/pix}}	Moodle icon helper — outputs an icon

5.2 How to Override a Template

1. Find the template you want to change in **theme/boost/templates/**
2. Copy it into your theme at the same relative path
3. Edit your copy — Moodle will use yours instead

```
# Example: override the navbar
docker exec moodle_app bash -c "
  cp /var/www/html/theme/boost/templates/navbar.mustache \
    /var/www/html/theme/ranees/templates/navbar.mustache
"

# Then purge caches
docker exec moodle_app php /var/www/html/admin/cli/purge_caches.php
```

■ *Only copy templates you actually need to change. Unused copies become maintenance debt — future Moodle upgrades may change the original template's context, breaking your override.*

5.3 Key Templates to Know

Template	Controls
drawers.mustache	Main page layout — header, sidebars, content area, footer
navbar.mustache	Top navigation bar — logo, primary nav, user menu
login.mustache	Login page HTML structure
footer.mustache	Page footer — links, Moodle version text
head.mustache	The HTML <head> — meta tags, CSS links
columns1.mustache	Single-column layout (popup pages)

columns2.mustache	Two-column layout (legacy)
drawer.mustache	Left sidebar drawer (course index)
blocks-drawer.mustache	Right sidebar drawer (blocks)
navbar-secure.mustache	Navbar for quiz/secure pages

5.4 Example — Custom Navbar Template

```

{{! theme/ranees/templates/navbar.mustache }}
<nav class="navbar navbar-expand navbar-dark">
  <div class="container-fluid">

    {{! Logo / Site name }}
    <a class="navbar-brand" href="{{config.wwwroot}}">
      
    </a>

    {{! Primary navigation }}
    {{#primarymoremenu}}
      {{> core/moremenu}}
    {{/primarymoremenu}}

    {{! Right side – user menu, notifications }}
    <div class="ml-auto d-flex align-items-center">
      {{#usermenu}}
        {{> core/user_menu}}
      {{/usermenu}}
    </div>

  </div>
</nav>

```

5.5 Moodle Template Helpers

```

{{! Output a language string }}
{{#str}} welcome, core {{/str}}

{{! Output a string with parameter }}
{{#str}} greetings, moodle, {{username}} {{/str}}

{{! Output a pix icon }}
{{#pix}} i/course, core, Course icon {{/pix}}

{{! Format a date }}
{{#userdate}} {{time}}, {{#str}} strftimedatefullshort, core_langconfig {{/str}} {{/userdate}}

{{! Output user picture }}
{{> core/user_picture}}

{{! Conditional block }}
{{#loggedin}}

```

```
        <p>Welcome back, {{fullname}}!</p>
    {{/loggedin}}
    {{^loggedin}}
        <p><a href="{{loginurl}}">Sign in</a></p>
    {{/loggedin}}
```

6. PHP Renderer Overrides

6.1 What is core_renderer?

Some HTML in Moodle is generated directly by PHP renderer classes, not Mustache templates. The main one is **core_renderer** in **lib/outputrenderers.php**. It handles things like form elements, notification boxes, user avatars, and navigation.

To override these, you extend the renderer in your theme's **classes/output/core_renderer.php**.

6.2 Creating the Renderer Override

```
<?php
// theme/ranees/classes/output/core_renderer.php

namespace theme_ranees\output;

use moodle_url;

defined('MOODLE_INTERNAL') || die();

class core_renderer extends \theme_boost\output\core_renderer {

    /**
     * Override the user avatar to add a custom CSS class.
     */
    public function user_picture(\stdClass $user, array $options = null) {
        $options = (array) $options;
        $options['class'] = ($options['class'] ?? '') . ' ranees-avatar';
        return parent::user_picture($user, $options);
    }

    /**
     * Add a custom banner above the main content on the dashboard.
     */
    public function main_content() {
        global $PAGE;
        $output = '';
        if ($PAGE->pagetype === 'my-index') {
            $output .= '<div class="ranees-welcome-banner">';
            $output .= '<h2>' . get_string('welcome', 'theme_ranees') . '</h2>';
            $output .= '</div>';
        }
        $output .= parent::main_content();
        return $output;
    }
}
```

■ The class must be in the correct namespace (*theme_ranees\output*) and the file must be at *classes/output/core_renderer.php* exactly. Moodle's autoloader finds it by convention.

7. JavaScript — AMD Modules

7.1 How Moodle's AMD System Works

Moodle uses RequireJS with AMD (Asynchronous Module Definition). You write ES modules in **amd/src/** and they get compiled to **amd/build/** by Grunt. In development (with Theme Designer Mode on), Moodle can serve the source files directly.

7.2 Creating a JS Module

```
// theme/ranees/amd/src/greeting.js

define(['jquery', 'core/log'], function($, log) {

    return {
        init: function(username) {
            log.debug('Ranees theme JS loaded for: ' + username);

            // Example: animate the welcome banner
            $('.ranees-welcome-banner').hide().fadeIn(600);

            // Example: custom dropdown behaviour
            $('.ranees-dropdown-toggle').on('click', function() {
                $(this).next('.ranees-dropdown-menu').toggleClass('show');
            });
        }
    };
});
```

7.3 Loading a Module from a Template

Call your module from a Mustache template using the **data-module** pattern, or load it directly from a PHP layout file.

```
{{! In your Mustache template – pass data via JSON attribute }}
<div id="ranees-greeting"
    data-username="{{username}}"
    data-module="theme_ranees/greeting">
</div>

// Or load from PHP layout file (layout/drawers.php)
$PAGE->requires->js_call_amd('theme_ranees/greeting', 'init', [$USER->firstname]);
```

7.4 Using Third-Party Libraries

You have two options for adding third-party JS libraries:

- **Option A — Copy into amd/src/:** Place the library file in **amd/src/** and require it as a local module. Good for versioning with your theme.

- **Option B — CDN via head.mustache:** Add a `<script>` tag in your head.mustache override. Simpler for libraries with CDN support.

```
{{! Option B – head.mustache: load Alpine.js from CDN }}  
<head>  
  {{> theme_boost/head}}  
  <script defer src="https://cdn.jsdelivr.net/npm/alpinejs@3/dist/cdn.min.js"></script>  
</head>
```

■ *For production, prefer Option A (self-hosted) so your site works offline and isn't dependent on external CDN availability.*

8. Development Workflow (Docker Setup)

8.1 Enable Theme Designer Mode

Theme Designer Mode disables CSS/template caching so changes appear immediately without a manual purge. Enable it at:

Site administration → Appearance → Themes → Theme settings → Theme designer mode

■ *Disable Theme Designer Mode on production — it significantly slows down page loads.*

8.2 Purging Caches

After any change to SCSS, templates, or PHP files, you must purge Moodle's caches.

```
# Via CLI (fastest – use this during development)
docker exec moodle_app php /var/www/html/admin/cli/purge_caches.php

# Via browser
# Go to: http://localhost:8080/admin/purgecaches.php
```

8.3 Mounting Your Theme as a Docker Volume

Instead of copying files into the container every time, mount your theme directory as a volume. This is the recommended development setup.

```
# docker-compose.yml – add a volume mount for your theme
services:
  moodle:
    volumes:
      - moodledata:/var/moodledata
      - ../theme/ranees:/var/www/html/theme/ranees # ← add this line
```

Now any file you edit locally is instantly available inside the container. You only need to purge caches, not rebuild the image.

8.4 Recommended Development Loop

Action	Command
Edit SCSS or template	Edit file locally (volume-mounted)
Purge caches	<code>docker exec moodle_app php /var/www/html/admin/cli/purge_caches.php</code>
Reload browser	Ctrl+Shift+R (hard reload to bypass browser cache)
Check PHP errors	<code>docker logs moodle_app</code>
Rebuild image	<code>docker compose up -d --build</code> (only after Dockerfile changes)

8.5 Useful Admin URLs

URL	Purpose
-----	---------

/admin/index.php	Site administration home
/admin/purgecaches.php	Purge all Moodle caches
/theme/index.php	Theme selector
/admin/settings.php?section=themesettings	Theme designer mode toggle
/admin/settings.php?section=theme_ranees	Your theme's admin settings page

9. Common Mistakes

X Editing core theme files directly

Any change inside theme/boost/ will be overwritten by the next Moodle upgrade. Always create a child theme and override only what you need.

X Forgetting to purge caches

Moodle aggressively caches SCSS, templates, and language strings. If your change isn't showing, run:
`php admin/cli/purge_caches.php`

X Wrong theme name casing in config.php

`$THEME->name` must exactly match the folder name, all lowercase. 'Ranees' will fail; 'ranees' will work.

X Missing require_once in config.php

`config.php` must start with: `defined('MOODLE_INTERNAL') || die();` — without this, the file can be accessed directly.

X SCSS syntax errors causing blank pages

A single SCSS error causes the entire stylesheet to fail, resulting in an unstyled page. Check docker logs `moodle_app` for the PHP error message.

X Wrong namespace in core_renderer.php

The class must be in namespace `theme_ranees\output`; and named `core_renderer`. Wrong namespace = Moodle won't find it.

X Overriding templates without tracking upstream changes

When you copy a template to override it, it no longer receives upstream fixes. After each Moodle upgrade, diff your overrides against the originals.

X Using {{{triple braces}}} on untrusted data

Triple braces output raw HTML without escaping. Only use it on data you fully control — never on user input.

10. Quick Reference

Minimal config.php

```
<?php
defined('MOODLE_INTERNAL') || die();

$THEME->name          = 'ranees';
$THEME->parents        = ['boost'];
$THEME->sheets         = [];
$THEME->extrascsscallback = 'theme_ranees_get_extra_scss';
$THEME->prescsscallback  = 'theme_ranees_get_pre_scss';
$THEME->scss_compiler_class = 'theme_boost\\autoprefixer';
$THEME->rendererfactory   = 'theme_overridden_renderer_factory';
$THEME->layouts = [/* copy from boost/config.php */];
```

Minimal lib.php

```
<?php
defined('MOODLE_INTERNAL') || die();

function theme_ranees_get_pre_scss($theme) {
    global $CFG;
    return file_get_contents($CFG->dirroot . '/theme/ranees/scss/pre.scss');
}

function theme_ranees_get_extra_scss($theme) {
    global $CFG;
    return file_get_contents($CFG->dirroot . '/theme/ranees/scss/post.scss');
}
```

Minimal version.php

```
<?php
defined('MOODLE_INTERNAL') || die();

$plugin->version      = 2024042800;
$plugin->requires      = 2023042400;
$plugin->component     = 'theme_ranees';
```

Minimal lang/en/theme_ranees.php

```
<?php
$string['pluginname'] = 'Ranees EdTech';
$string['choosereadme'] = 'Custom theme for Ranees EdTech platform.';
```

CLI Command Reference

Command	Purpose
php admin/cli/purge_caches.php	Purge all Moodle caches
php admin/cli/cfg.php --name=theme --set=ranees	Set active theme via CLI

<code>php admin/cli/cfg.php --name=themedesignermode --set=1</code>	Enable designer mode
<code>php admin/cli/upgrade.php</code>	Run Moodle upgrade (after version bump)
<code>php admin/cli/install_database.php</code>	Initial DB install (first run)

For questions and issues, refer to the official Moodle documentation at **docs.moodle.org** and the Moodle Developer documentation at **moodledev.io**.